

AI 100 講 — 補充單元

VS Code IDE

看著 AI 在你旁邊工作

它能問、能改、能想、能修、能造

按 → 或空白鍵開始

Python

```
import pandas as pd
def analyze_sequence(raw_data):
    dat = 2
    for se_data in data:
        if data[se_data] > 0.8:
            analyze_sequence(raw_data)
    return data
```

CSS

```
body {
  background-color: #0000ff;

  color: 100%;
  color: -1 0;
}
```

Rust

```
fn calculate_velocity(m: f64, v: f64) {
  fn f6e vee 2 {
    int f6e vee 0;
    return (m * v);
  }
}
```

C++

```
const int slots
const int veeerr: "on"
const int veeerr: "vandy"
const int vertlase = (vandy: f6e vee 0);

for (int i=0; i<vandy) {
  int beti;
  printf("%d", vee vee vee);
  printf("%d", f6e vee);
  return (vandy: f6e vee 0);
}
```

HTML

```
<div class="container">
  <header>Navigation</header>
</div>
```

CSS

```
body {
  background-color: #0000ff;
}
```

JavaScript

```
function main() {
  fetch(api/v1/data).then(...)
    .then(a => {
      return(f6e vee);
    });
}
```

ZERO
GRAVITY
COFFEE

今天的旅程

- 1 為什麼需要 IDE + 安裝**
CLI 跟 IDE 的差別，裝好 Claude 擴充
- 2 五個遞進案例**
AI 能力從「回答」到「完整改造」
- 3 四大 AI 擴充比較**
Claude / Copilot / Amazon Q / Continue
- 4 工具全景 + 收尾**
S01 ~ S05 什麼時候用什麼工具

S03 建造者 vs S04 改造者

S03

建造者

空資料夾 → 全新專案

從零開始

CLI 打字指揮

看文字輸出

S04

改造師

現有專案 → 客製產品

接手改造

IDE 可視化操作

每步都能 diff

90% 的工作不是從零開始，是**接手 + 改造**

為什麼需要 IDE ？

CLI 模式

像跟 AI 打電話

- 看文字跑，看不到全貌
- 改了什麼？不確定
- 多檔案？很難追蹤
- Debug？得自己切畫面

IDE 模式

像 AI 坐你旁邊

- 看到所有檔案結構
- 每一步都能 diff 對照
- 檔案樹一目了然
- Console 錯誤直接貼

什麼時候用 IDE ？

接手別人的專案、多檔案修改、需要 debug、看程式碼結構

額度確認 **FIRST**

裝完才發現沒額度 = 浪費時間



Claude Pro \$20/月

今天的課夠用。推薦。



沒訂閱？

Amazon Q（免費 50 次/月）或 Continue.dev（免費，接任何模型）

先確認，不要裝完才發現沒額度

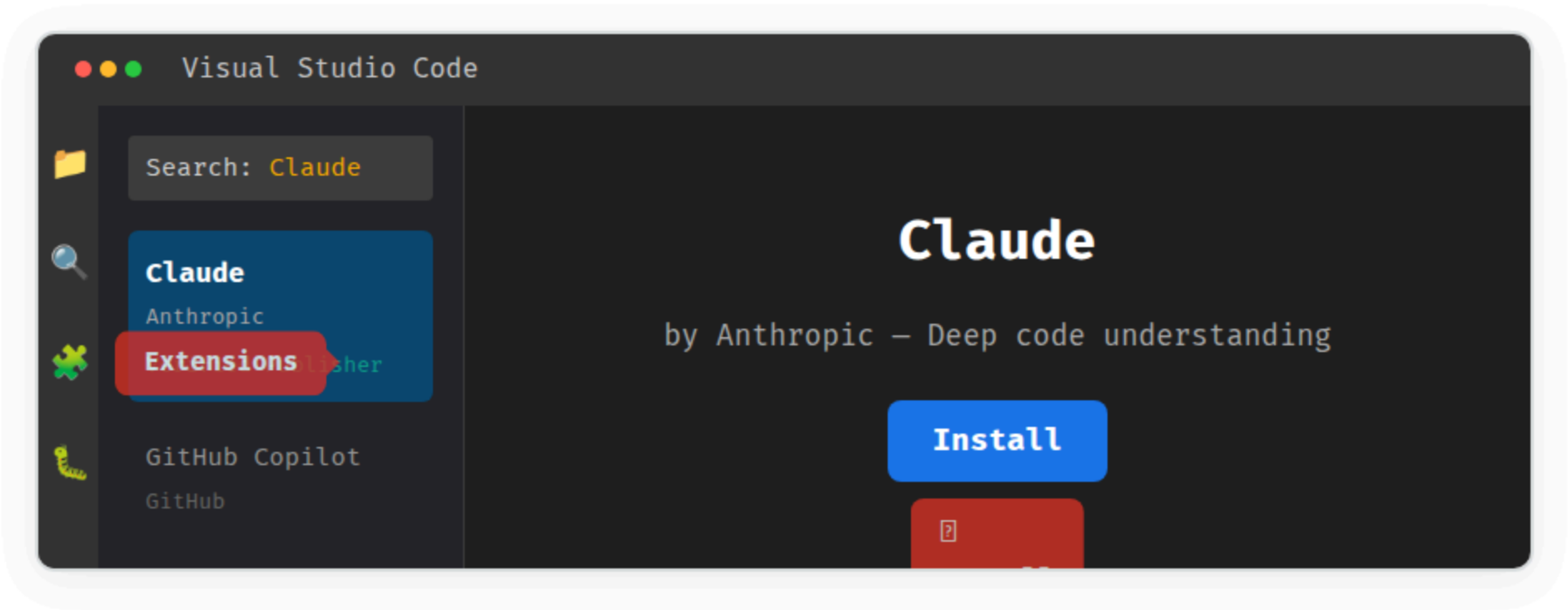
打開 claude.ai → 看右上角是否有 Pro / Max 標示

沒有的話，等等用免費替代方案也能跟上



全班跟做

安裝 Claude for VS Code



全班跟做

登入帳號

1 安裝完成 → 側邊欄出現 Claude 圖示

點擊它打開聊天面板

2 Sign In with Claude.ai

用你的 Claude 帳號登入

3 看到聊天框 = 成功

可以開始跟 AI 對話了

確認成功

在聊天框打「hello」→ Claude 回覆 → 你準備好了



AI 回答問題

"它看懂了"

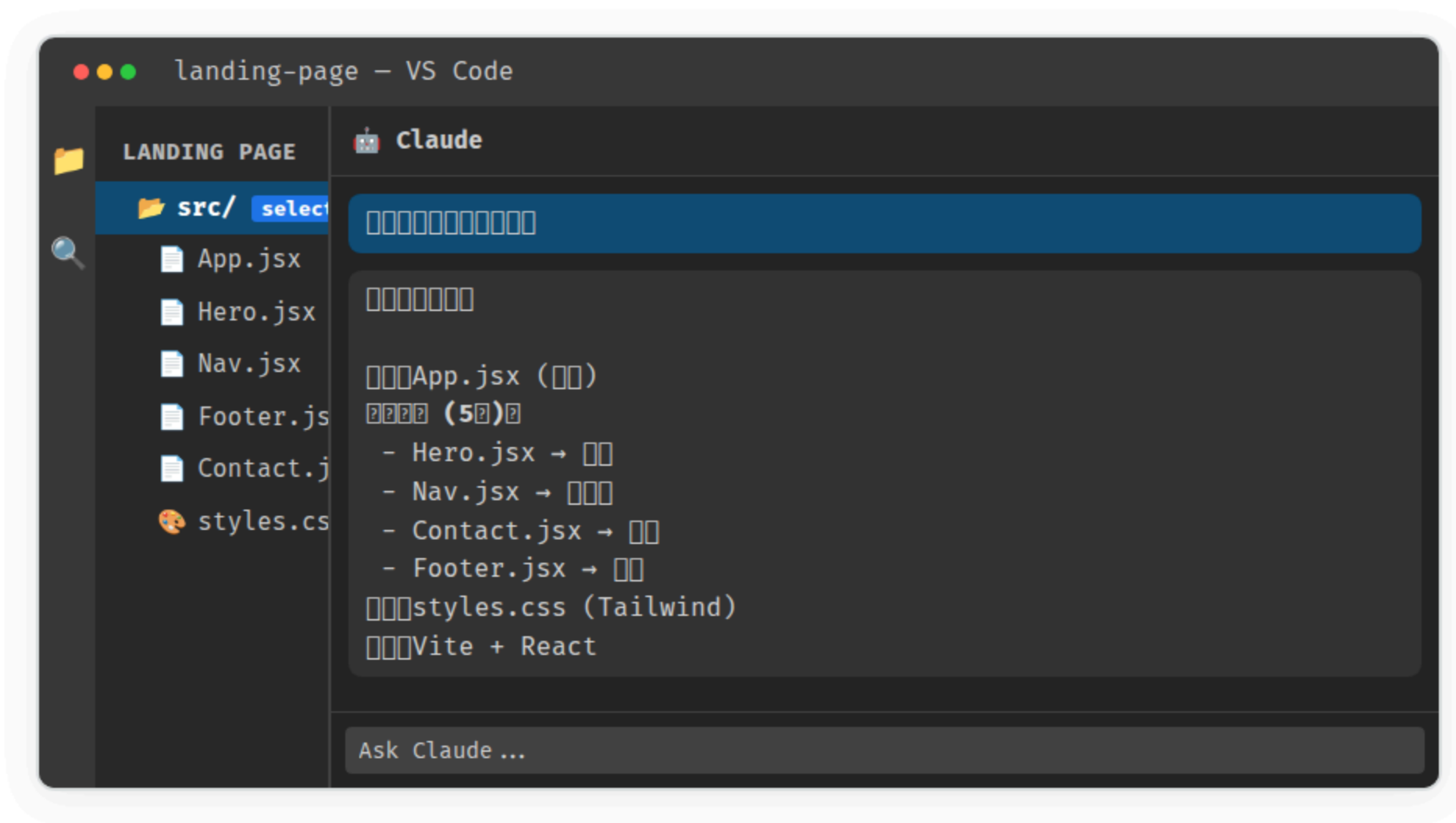
Demo + 跟做

選一段問它：「這段做什麼？」



Demo + 跟做

問整個專案：「架構是什麼？」



選一段問它，它真的看懂了。選整個資料夾，它看懂架構。



全班跟做

打開任一本地檔案 → 選取 → 問 Claude

1 File → Open Folder

打開你電腦上任何一個專案或資料夾

2 選取一段程式碼

任何檔案都行 — HTML、CSS、JS、Python...

3 在 Claude 面板問：「這段做什麼？」

或：「這個專案的架構是什麼？」

感受

「選一段問它，它真的看懂了」



AI 幫我改

"8 個檔案都改了"

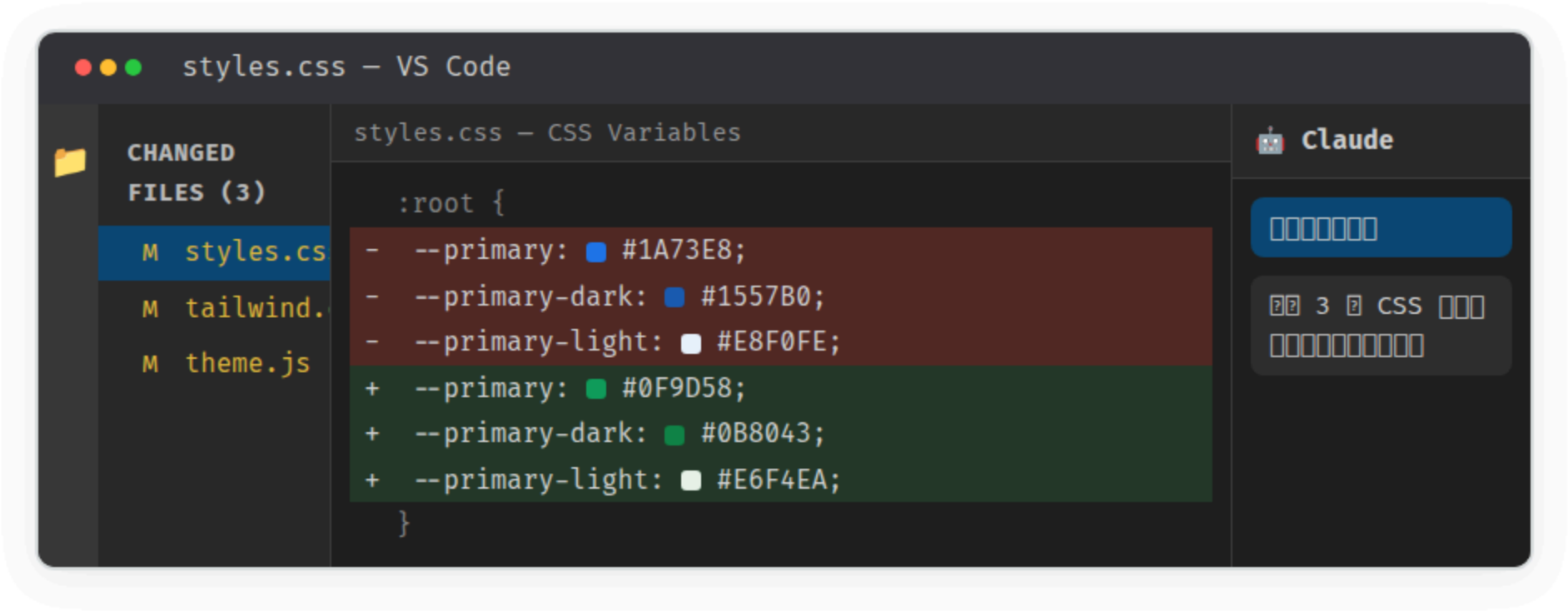
Demo

「把英文介面改成繁體中文」



Demo

「配色從藍改成綠」



我說改中文，8 個檔案都改了，每一步我都能看到

Diff View 的價值

沒有 Diff

AI 說：「改好了」
你：「改了什麼？」
AI 說：「信我就對了」



有 Diff

紅色 = 刪掉的

綠色 = 新增的

你看得到每一個字的變化

Accept / Reject

IDE 最大的優勢：**每一步都透明**



AI 替我想

"它自己決定改哪"

二星 vs 三星的差別

二星：你指定改哪裡

「把 英文 改成 中文」

「把 藍色 改成 綠色」

你告訴 AI 改什麼 + 怎麼改

三星：你只說要什麼

「加一個深色模式」

「加一個搜尋功能」

AI 自己決定改哪些檔案

關鍵轉變

你不需要告訴 AI 「改第 42 行」，你只需要說「我要這個功能」

AI 自己分析該動哪些檔案、該新增什麼

互動 Demo

你們想加什麼功能？

講師打開那個 Landing Page 專案，現場問大家



深色模式

切換深色/淺色主
題



搜尋功能

在網站內搜尋內
容



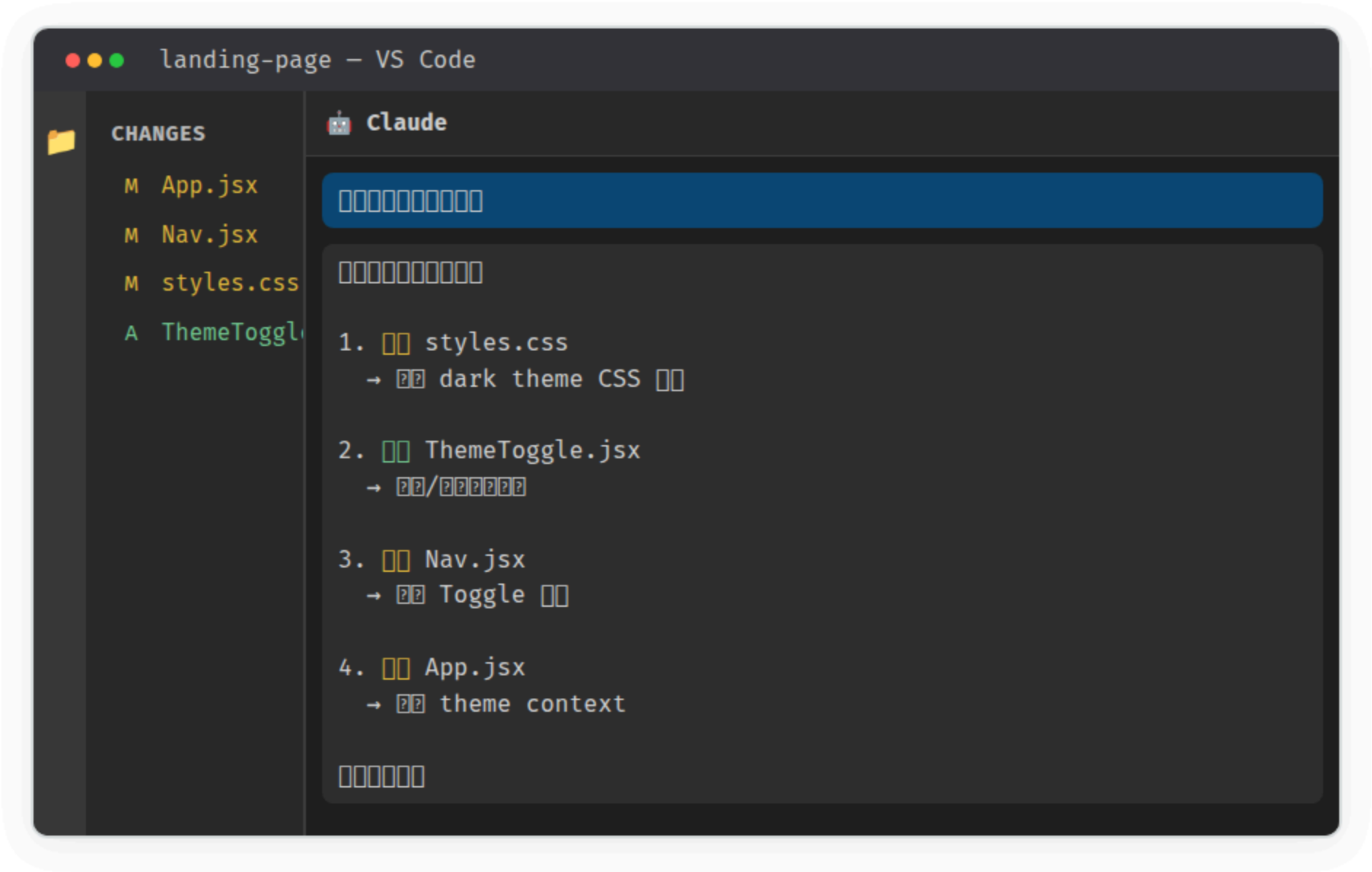
多語系

中/英語言切換

現場投票 → 講師輸入 → Claude 開始規劃

Demo

Claude 的規劃



我只說要什麼，它自己決定怎麼做

Apply → Preview 結果



三星的感受

「我只說要什麼，它自己決定改哪」

跟二星的差異

二星：你說「把這邊改成那樣」（你指路）

三星：你說「我要這個效果」（AI 自己找路）



休息 10 分鐘

接下來：AI 幫你找 Bug + 完整改造案例

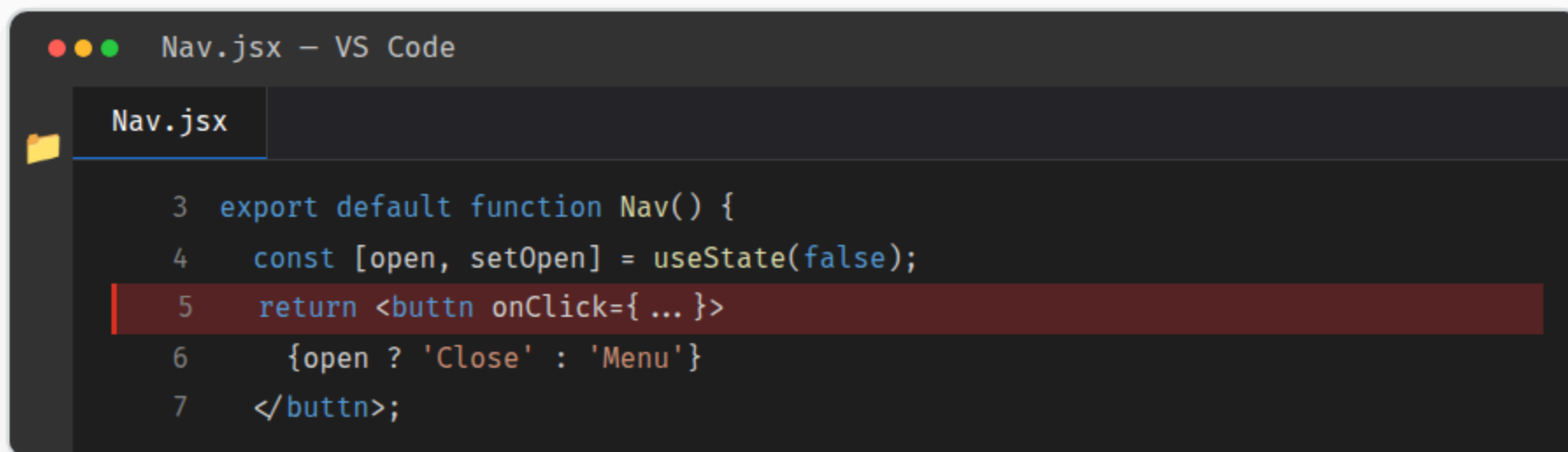


AI 找問題

"它比我先找到"

Demo

講師故意搞壞一行 code



```
3 export default function Nav() {  
4   const [open, setOpen] = useState(false);  
5   return <buttn onClick={ ... }>  
6     {open ? 'Close' : 'Menu'}  
7   </buttn>;
```

打開網頁 → **按鈕消失了！**

「如果你，你怎麼辦？」

Demo

Console Error → 貼給 Claude



一鍵修好 → 網頁恢復



Debug 的正確姿勢

你不需要看懂 code

你需要知道：**怎麼問 AI 正確的問題**

Error → Copy → Ask → Fix → Verify

這是你以後遇到任何 bug 的標準流程

為什麼架構很重要？

沒有架構

所有程式碼塞一個檔案
AI 想加功能 → 不知道放哪
越改越亂 → 義大利麵條



好的架構

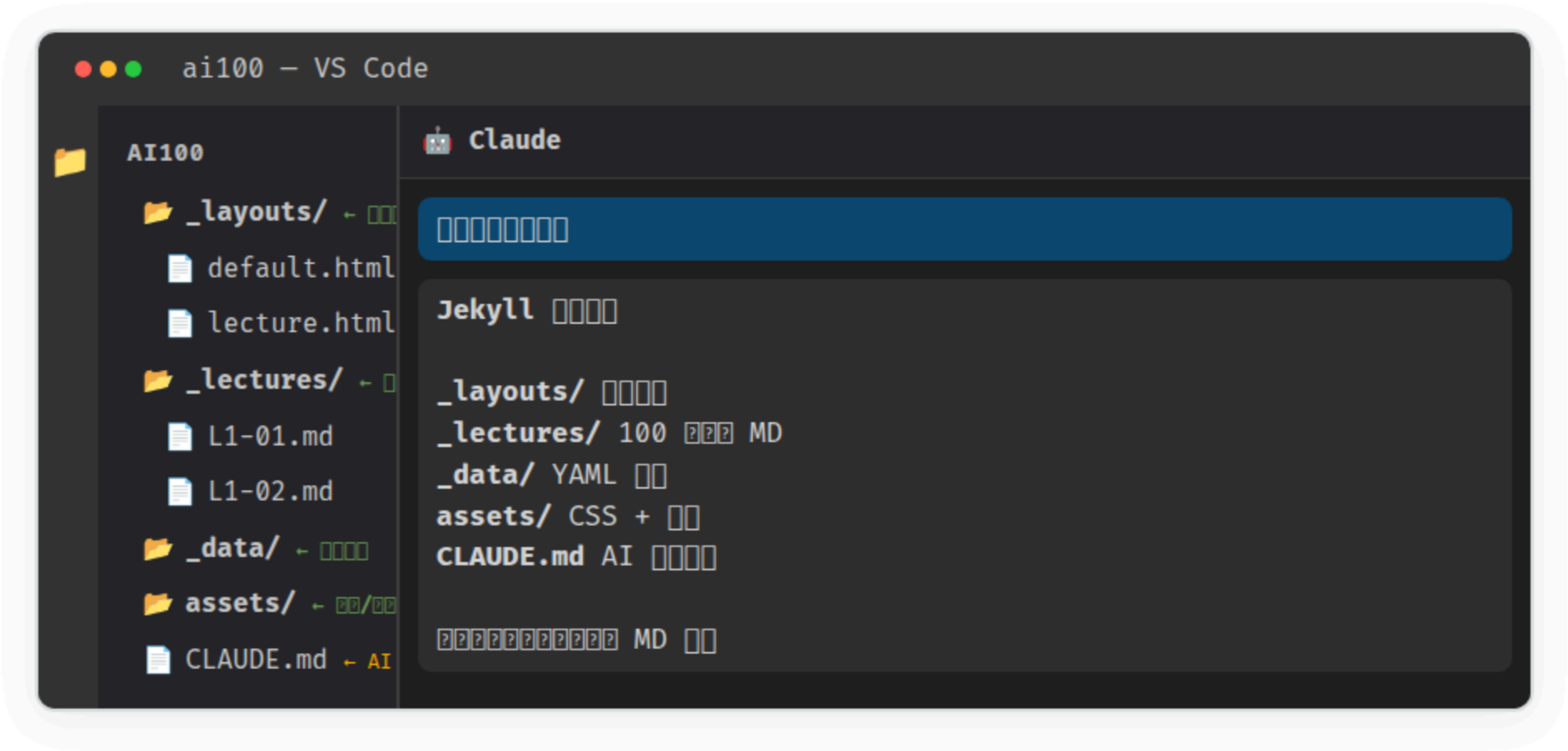
每個檔案有明確職責
AI 知道新功能放哪裡
改一個不會壞其他的



S03 學的 CLAUDE.md 規則，在 IDE 裡一樣有用

CLAUDE.md = 給 AI 的工作手冊，告訴它專案的規則與架構

好架構的範例：AI 100 講網站





休息 10 分鐘

最後：完整改造大 Demo + 工具全景



完整改造

"開源模板變成我的產品"

大 Demo：咖啡店模板 → 山海咖啡

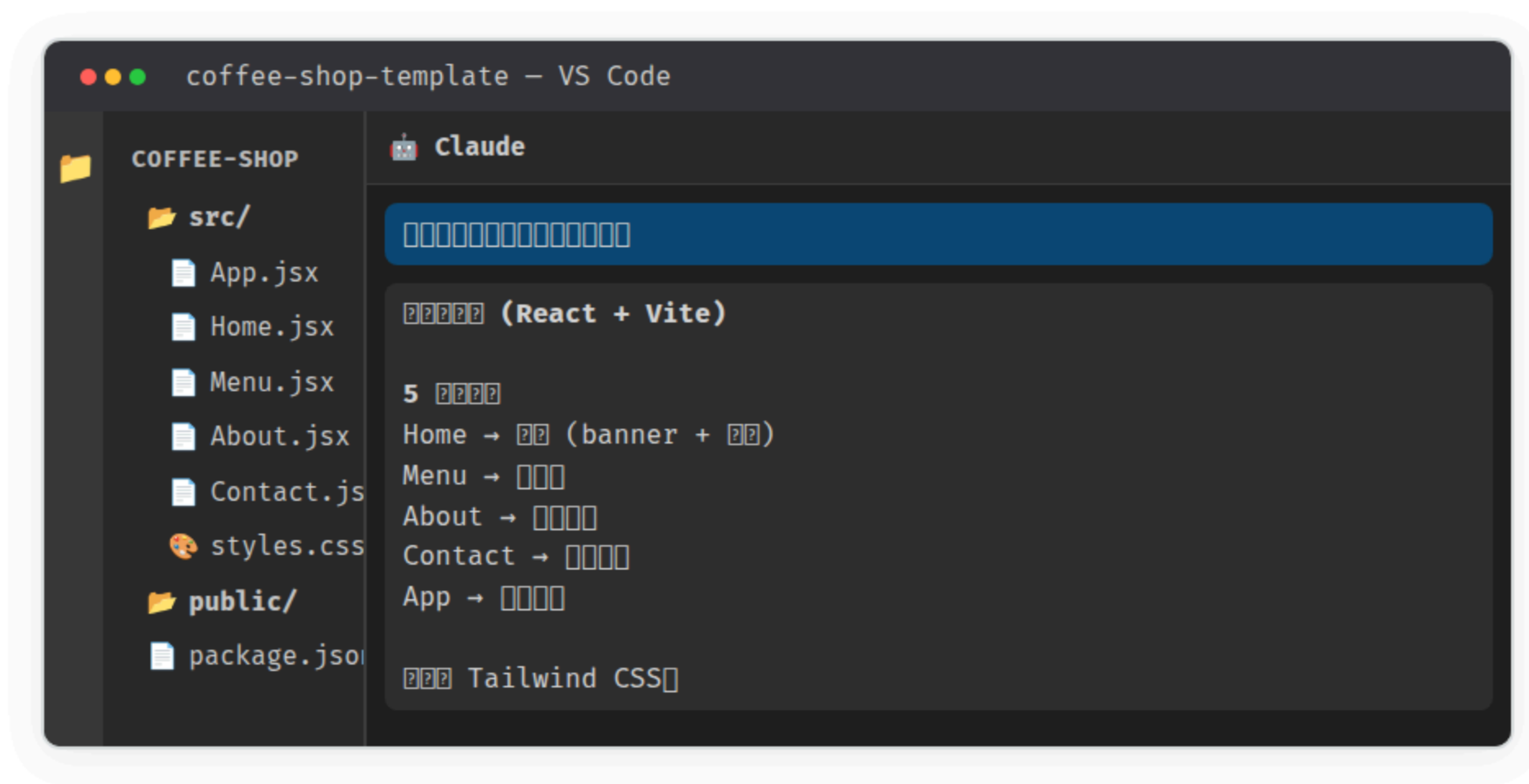
GitHub 上找到一個免費咖啡店模板，用五星改造流程把它變成自己的產品



這就是真實世界的工作方式：找模板 → 客製化 → 上線

Demo — Step 1

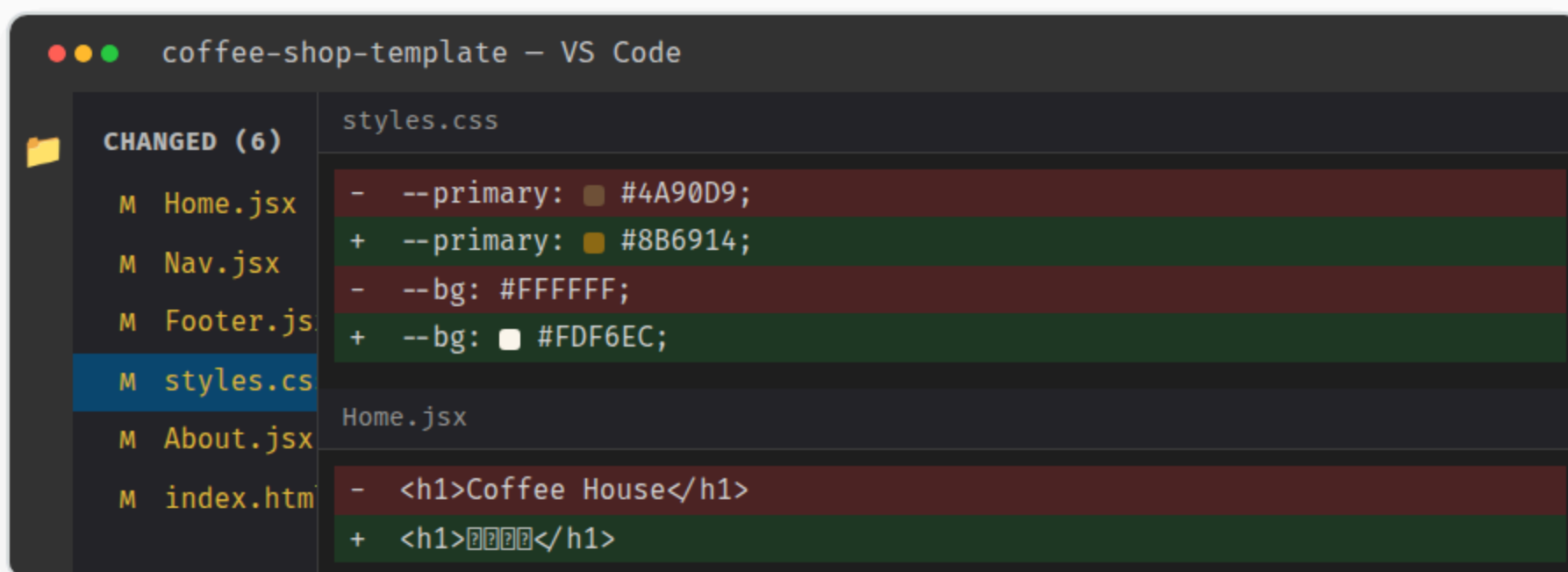
★ 找：Clone 模板 + 問架構



Demo — Step 2

★★ 改外觀：店名 + 配色

「店名改成山海咖啡，配色改成木質色系」



The screenshot shows the VS Code editor interface for a project named 'coffee-shop-template'. The left sidebar displays a file explorer with a list of files: Home.jsx, Nav.jsx, Footer.js, styles.cs, About.jsx, and index.htm. The 'styles.cs' file is selected and highlighted in blue. The main editor area shows the content of 'styles.css' and 'Home.jsx' with changes indicated by red minus signs and green plus signs.

```
coffee-shop-template - VS Code
```

CHANGED (6)

- M Home.jsx
- M Nav.jsx
- M Footer.js
- M styles.cs
- M About.jsx
- M index.htm

styles.css

```
- --primary: #4A90D9;  
+ --primary: #8B6914;  
- --bg: #FFFFFF;  
+ --bg: #FDF6EC;
```

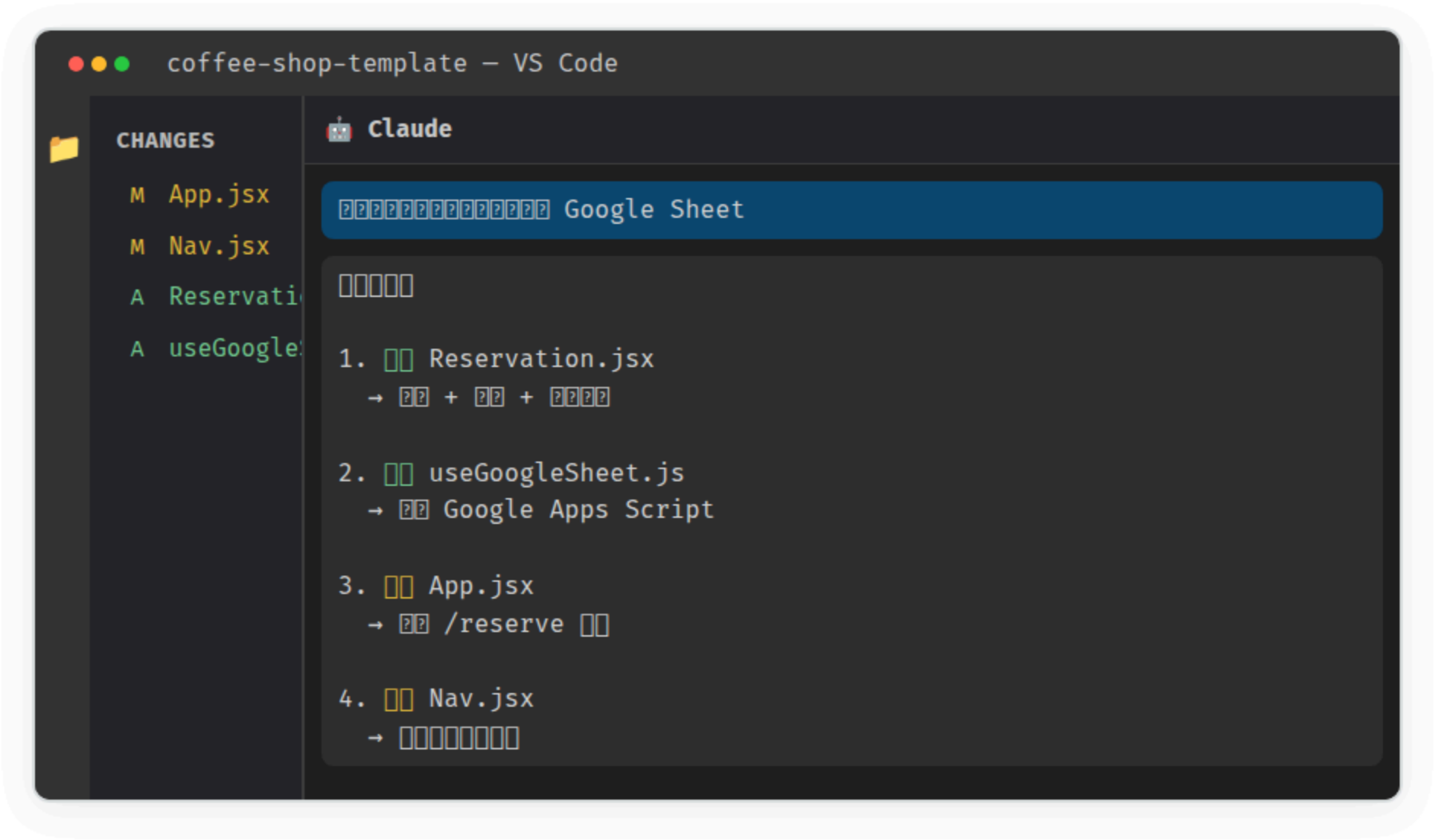
Home.jsx

```
- <h1>Coffee House</h1>  
+ <h1>山海咖啡</h1>
```


Demo — Step 3

★★★ 加功能：線上預約

講師問：「咖啡店還需要什麼？」 → 「線上預約！」



Demo — Step 4

★★★★ 修問題：表單壞了

故意搞壞預約表單 → 用四星流程修

表單送出 → 白畫面



F12 Console



複製 Error



Claude 修好

The screenshot shows the VS Code editor with a file named 'Reservation.jsx'. A diff view is active, showing a change from a simple fetch call to a fetch call with options. The code is as follows:

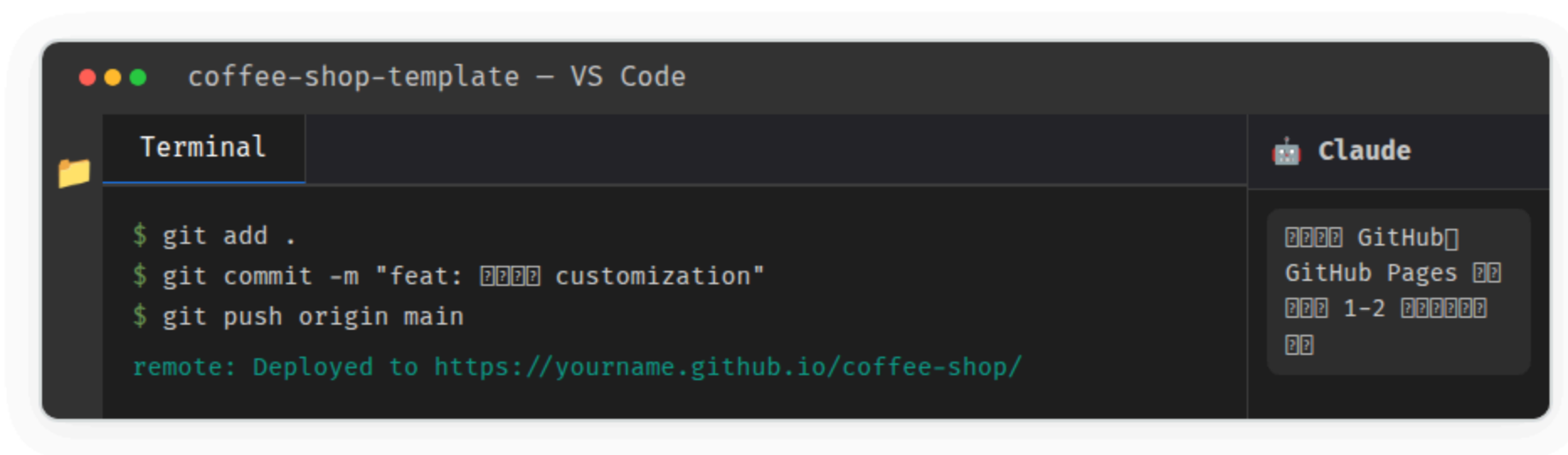
```
- const res = await fetch(SHEET_URL)
+ const res = await fetch(SHEET_URL, {
+   method: 'POST',
+   body: JSON.stringify(formData)
+ })
```

To the right of the code editor is the Claude AI chat interface. It shows a message from the user asking for help with a 'fetch' error, and a response from Claude suggesting to use 'POST' and provide a 'body'.

fetch ??? GET??
??? POST ???
body[]

Demo — Step 5

★★★★★ 部署到 GitHub Pages



The screenshot shows a VS Code window titled "coffee-shop-template - VS Code". It features a split view with a "Terminal" pane on the left and a "Claude" chat pane on the right. The terminal displays the following commands and output:

```
$ git add .  
$ git commit -m "feat: 新增 customization"  
$ git push origin main  
remote: Deployed to https://yourname.github.io/coffee-shop/
```

The Claude chat pane shows a conversation with the AI assistant:

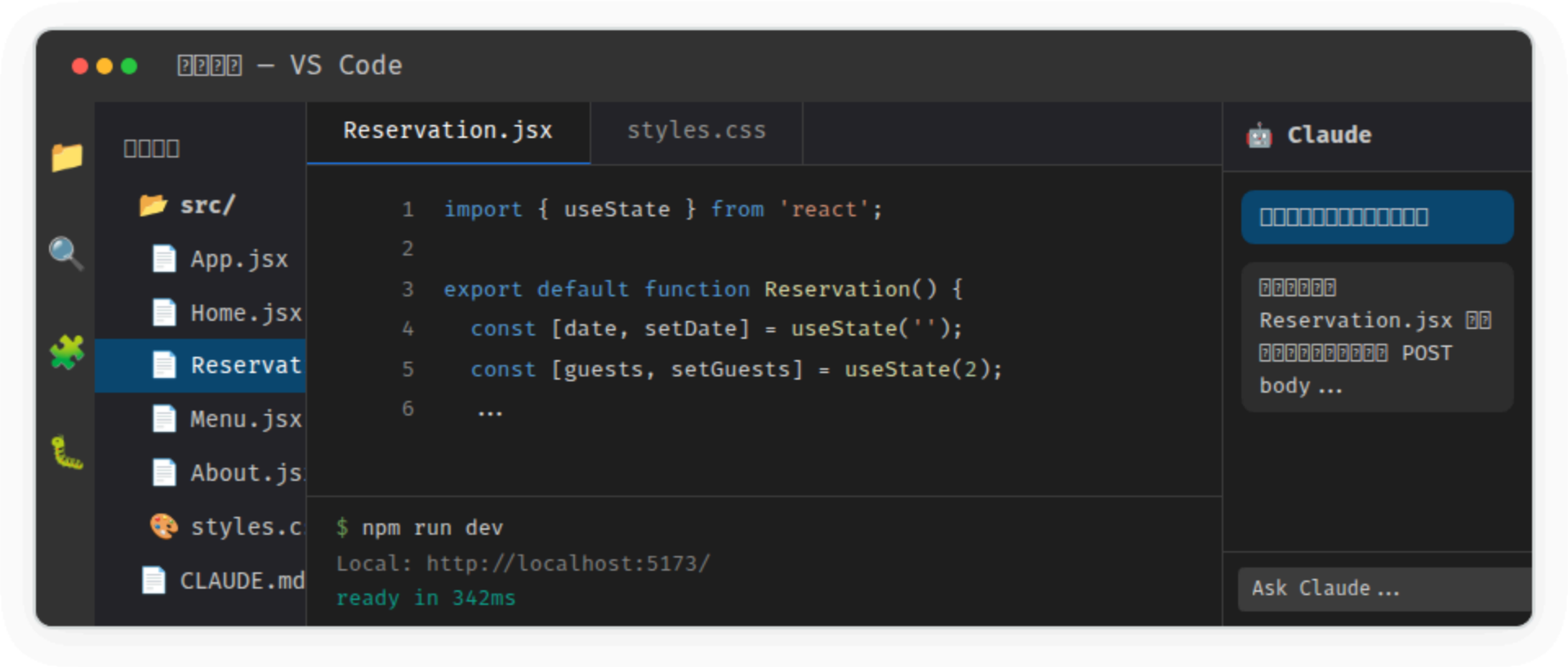
```
新增 GitHub  
GitHub Pages 新增  
新增 1-2 新增  
新增
```

完成！

一個開源模板 → 變成**山海咖啡**的專屬網站

找 → 改外觀 → 加功能 → 修問題 → 部署上線

VS Code 四合一



The screenshot shows the VS Code IDE interface with four main components: a file explorer on the left showing a project structure with a 'src/' folder containing 'App.jsx', 'Home.jsx', 'Reservat', 'Menu.jsx', and 'About.js', and a 'styles.c' file; a code editor in the center showing a 'Reservation.jsx' file with React code; a terminal at the bottom showing the command '\$ npm run dev' and the output 'Local: http://localhost:5173/ ready in 342ms'; and a Claude chat interface on the right with a text input field and a chat history showing a previous message about 'Reservation.jsx' and a 'POST' request.

檔案樹

+

程式碼

+

終端機

+

Claude 聊天

改造者 Workflow 總結



找 — 問架構

Clone 專案 → 問 Claude 「架構是什麼？」 → 理解全貌



改 — 指定修改

「改中文」「改配色」→ 你指路，AI 執行，diff 確認



加 — 說要什麼

「加預約功能」→ AI 自己規劃，你只看結果



修 — 貼 Error

Error → Copy → Ask → Fix → Verify



部署 — 推上線

git push → GitHub Pages → 全世界看到

工具比較

四大 AI 擴充

Claude / Copilot / Amazon Q / Continue

四大 AI 擴充比較

擴充	特色	價格	推薦指數
Claude for VS Code	深度理解、長上下文、可對話	\$20/月 (Pro)	★★★★★
GitHub Copilot	最快自動補全、程式碼建議	\$10/月 或免費版	★★★★★
Amazon Q	免費 50 次/月、安全掃描	免費	★★★★
Continue.dev	開源、接任何模型 (GPT/Claude/local)	免費	★★★★

最佳免費組合

Claude 延伸 + Copilot Free + Amazon Q = 三個都裝，互補使用
Claude 問問題、Copilot 自動補全、Amazon Q 安全掃描

什麼時候用什麼？

C 問問題、理解架構、多檔案修改
→ Claude for VS Code（深度理解最強）

G 寫程式碼、自動補全、一行一行寫
→ GitHub Copilot（速度最快）

A 安全掃描、免費額度
→ Amazon Q（完全免費）

O 想用自己的模型、本地 LLM
→ Continue.dev（開源彈性最大）

決策指南

工具全景決策樹

S01 ~ S05 什麼時候用什麼

你要做什麼？

→ Google 試算表自動化

GAS (S01) — AI 幫你寫腳本

→ 快速做網站 / 原型

Vibe Coding (S02) — 說話就能做

→ AI 操作你的電腦

CLI Agent (S03) — 從空資料夾開始

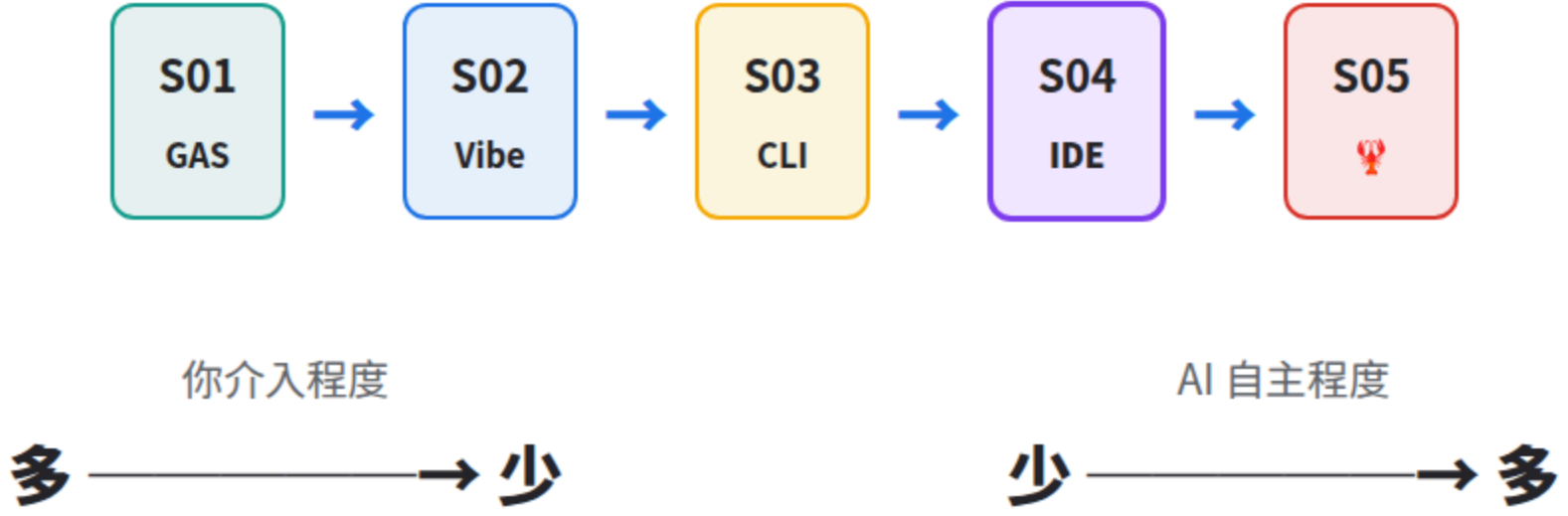
→ 開發 / 改造複雜專案

VS Code IDE (S04) — 接手 + 改造

→ 24/7 自動化 AI 管家

OpenClaw (S05) — AI 不下班

五堂課的進程



今天你學會了

★ AI 回答 — 選一段問它，它真的看懂了

★★ AI 改 — 8 個檔案都改了，每步都有 diff

★★★ AI 想 — 我只說要什麼，它自己決定改哪

★★★★ AI 修 — Error → Copy → Ask → Fix → Verify

★★★★★ AI 改造 — 開源模板變成自己的產品

帶走一句話

**90% 的工作
不是從零開始
是接手 + 改造**

你不需要會寫程式。你需要會**讀架構**、**看 diff**、**貼 error**。

下一堂預告

前四堂，AI 都是你開工具它才做事。

你關電腦，AI 就下班。

下一堂，AI 不下班了。



S04 補充單元

完成！

問 → 改 → 想 → 修 → 改造

改造者的五個層次

ai100.cooperation.tw